THREE SIGMA

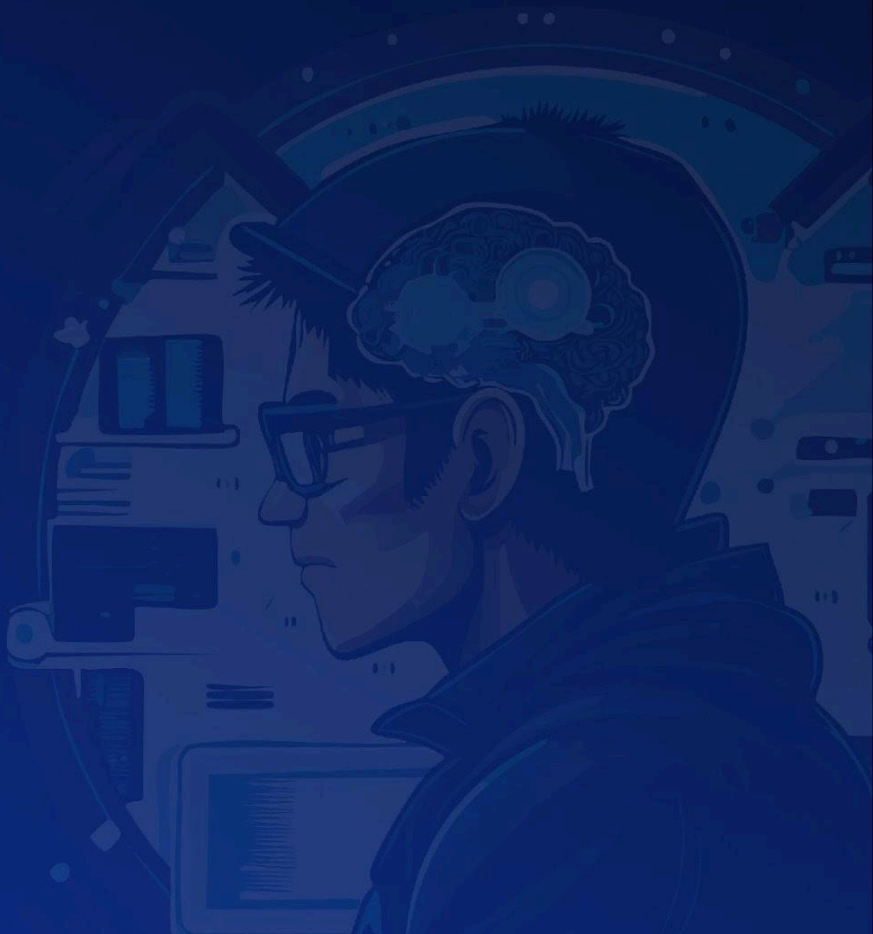
InfiniFi Protocol

# Security Review

# Disclaimer

## Security Review

InfiniFi Protocol



# Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

# Table of Contents

## Security Review

InfiniFi Protocol

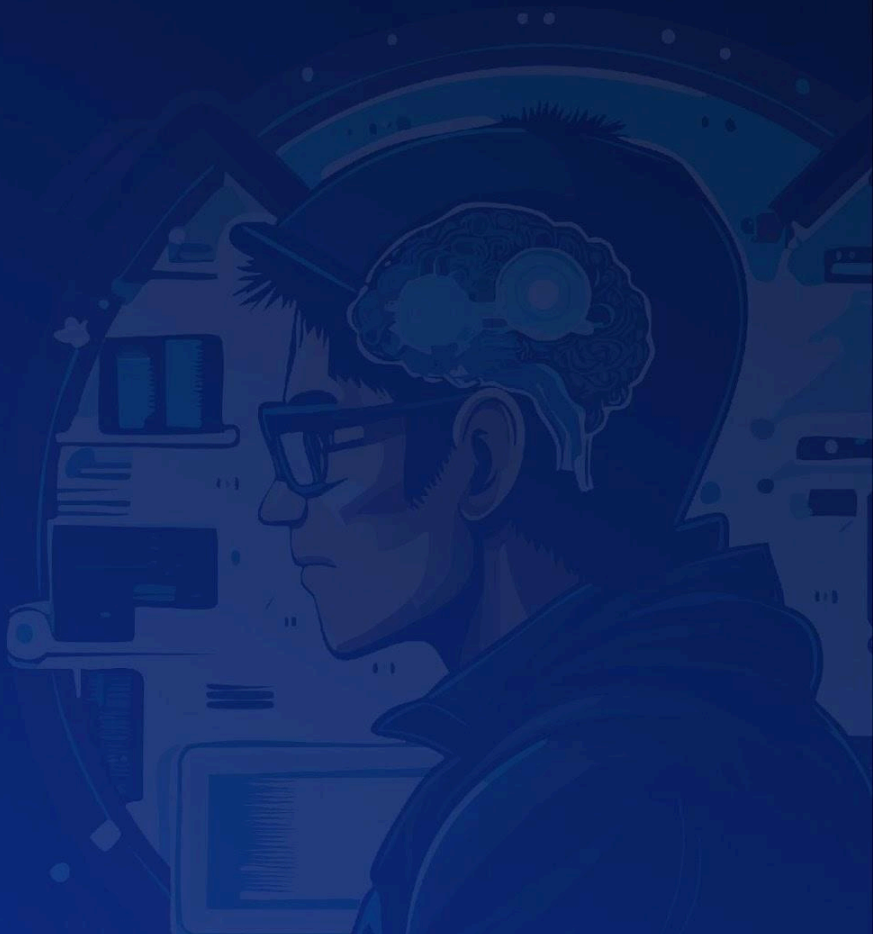
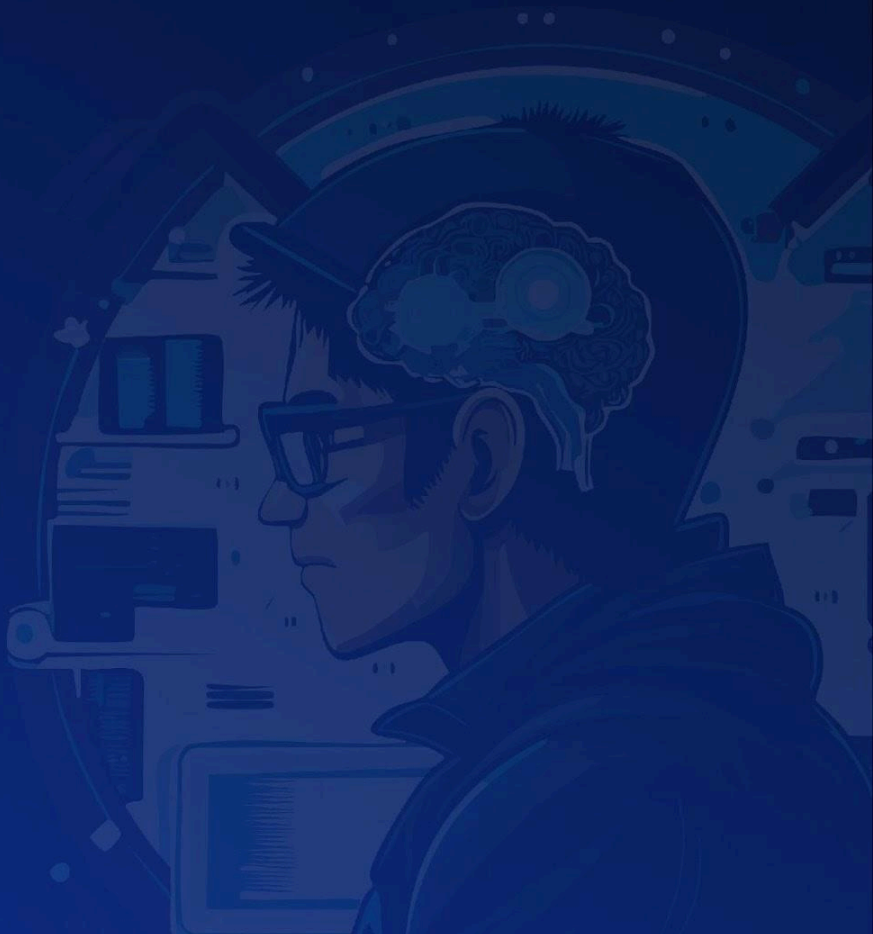


Table of Contents

# Summary

## Security Review

InfiniFi Protocol



# Summary

Three Sigma audited InfiniFi in a 0.8 person week engagement. The audit was conducted from 23/02/2026 to 24/02/2026.

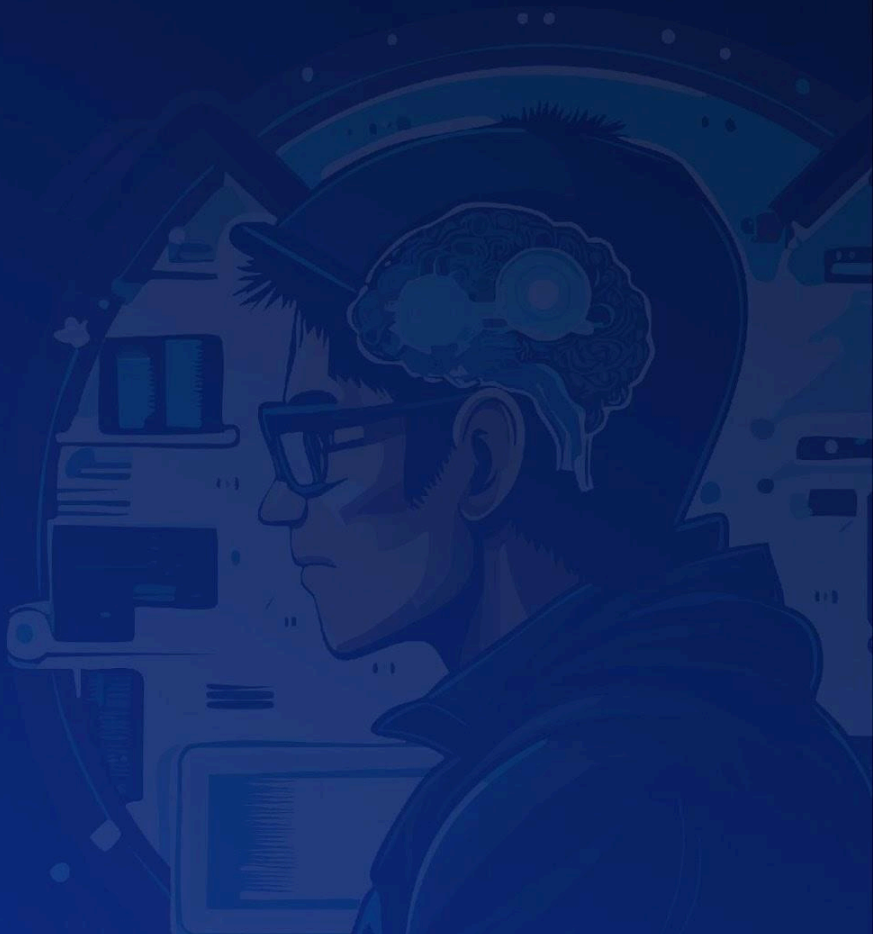
## Protocol Description

InfiniFi is a DeFi protocol that enables users to mint and redeem receipt tokens (iUSD, iETH) against collateral assets (USDC, ETH). The protocol features a sophisticated yield generation system through multiple farm integrations, a locking mechanism for enhanced rewards, and a governance system for farm allocation voting.

# Scope

## Security Review

InfiniFi Protocol



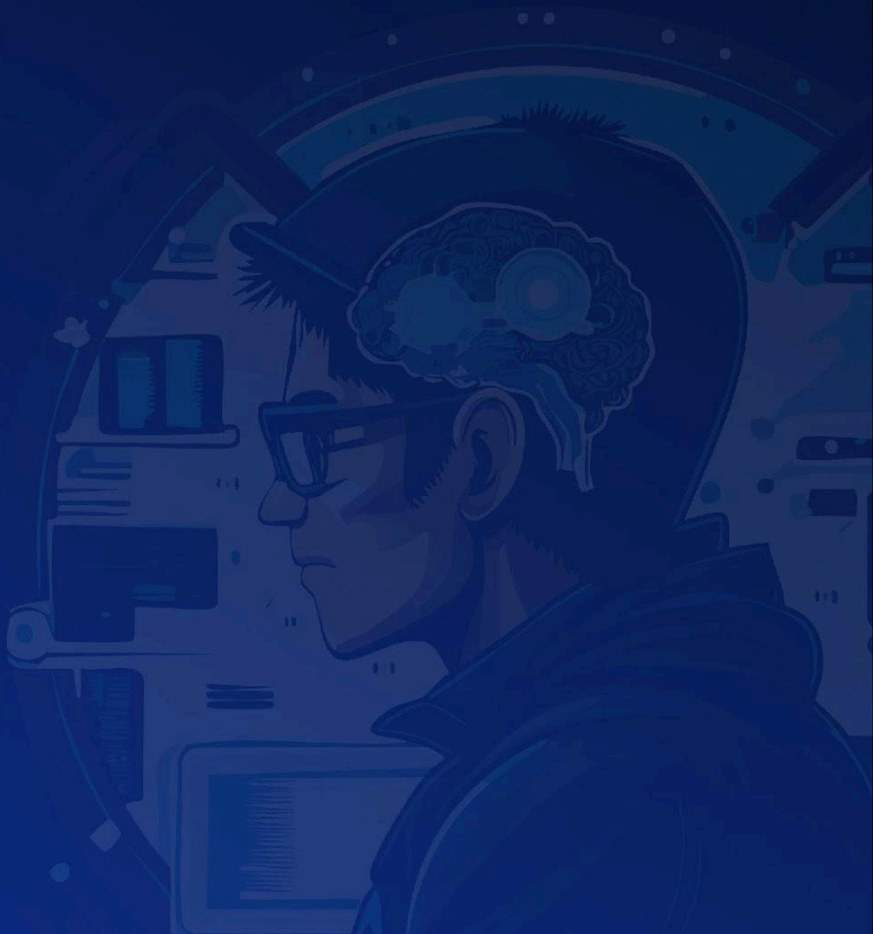
# Scope

| Filepath | nSLOC |
|---|---|
| R1:src/finance/PLSmoother.sol | 98 |
| R1:src/integrations/farms/LiquidationFarm.sol | 126 |
| R2:src/integrations/farms/UpshiftFarm.sol | 101 |
| R3:src/finance/RWAEscrowRateManager.sol | 43 |
| **Total** | **368** |

# Methodology

## Security Review

InfiniFi Protocol



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

## Taxonomy

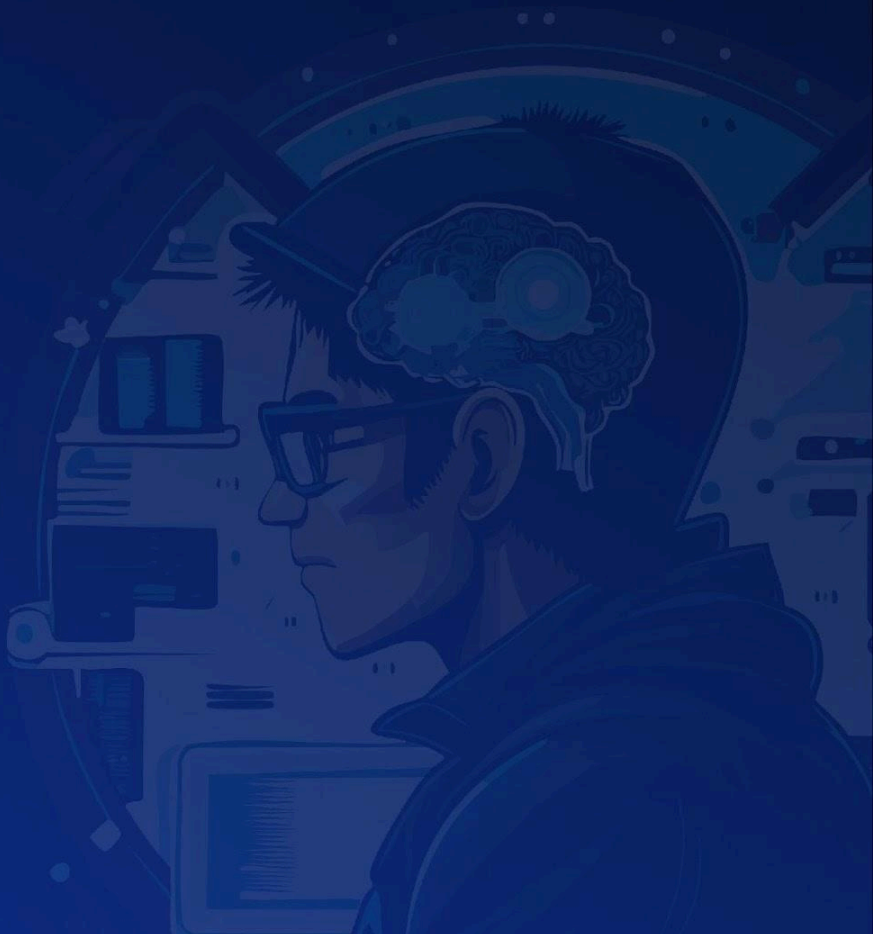
In this audit, we classify findings based on Immunefi's Vulnerability Severity Classification System (v2.3) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

| Impact / Likelihood | LOW | MEDIUM | HIGH |
|---|---|---|---|
| NONE | None | | |
| LOW | Low | | |
| MEDIUM | Low | Medium | Medium |
| HIGH | Medium | High | High |
| CRITICAL | High | Critical | Critical |

# Project Dashboard

## Security Review

InfiniFi Protocol



# Project Dashboard

## Application Summary

| | |
|---|---|
| Name | InfiniFi |
| Repository | R1:https://github.com/InfiniFi-Labs/infinifi-contracts/pull/312<br>R2:https://github.com/InfiniFi-Labs/infinifi-contracts/pull/314<br>R3:https://github.com/InfiniFi-Labs/infinifi-contracts/pull/315 |
| Commit | R1 - c44db35<br>R2 - 0b34cf4<br>R3 - 5bcc644 |
| Language | Solidity |
| Platform | EVM |

## Engagement Summary

| | |
|---|---|
| Timeline | 23/02/2026 to 24/02/2026 |
| № of Auditors | 2 |
| Review Time | 0.8 person weeks |

## Vulnerability Summary

| Issue Classification | Found | Addressed | Acknowledged |
|---|---|---|---|
| Critical | 0 | 0 | 0 |
| High | 0 | 0 | 0 |
| Medium | 1 | 1 | 0 |
| Low | 1 | 1 | 0 |
| None | 7 | 5 | 2 |

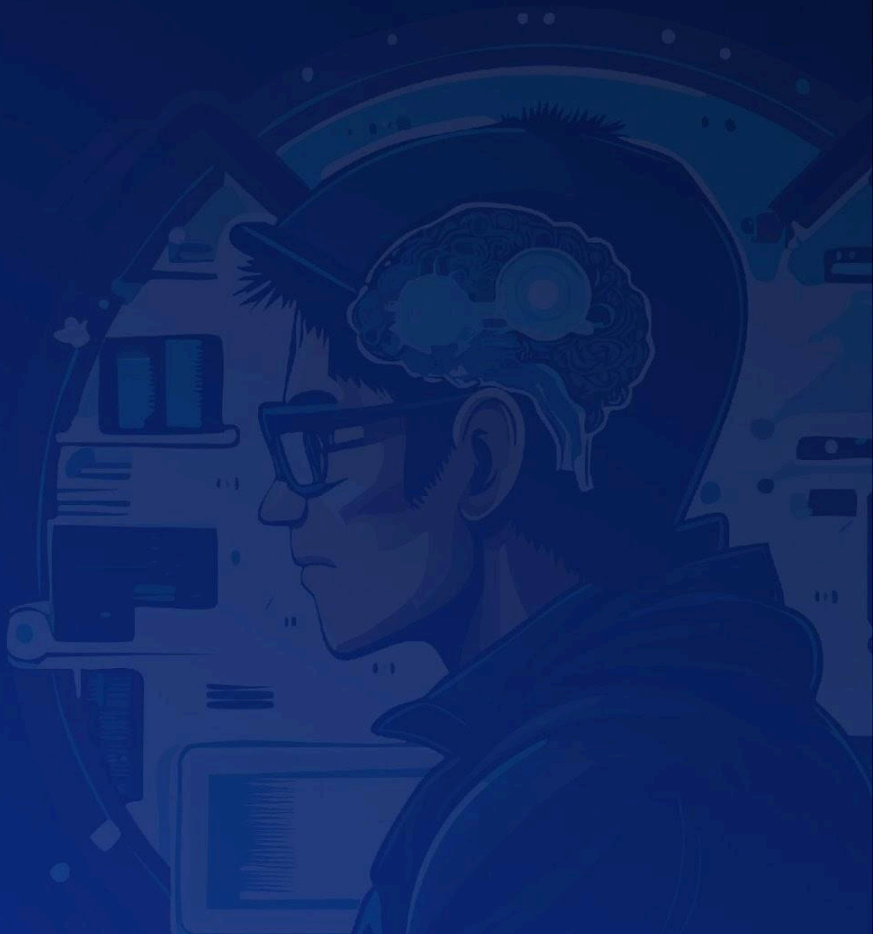
## Category Breakdown

| | |
|---|---|
| Suggestion | 7 |
| Documentation | 0 |
| Bug | 2 |
| Optimization | 0 |
| Good Code Practices | 0 |

# Risk Section

## Security Review

InfiniFi Protocol



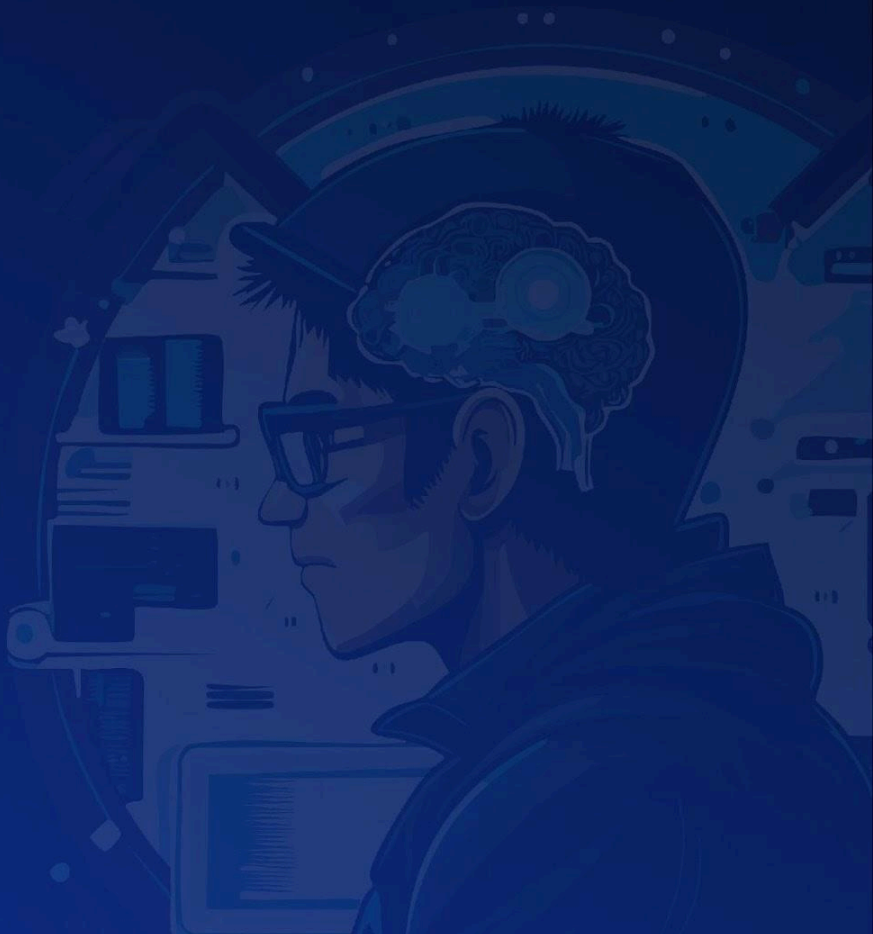
# Risk Section

- **Partial Audit Scope**: The audit covers only specified contracts within the repository. Vulnerabilities in contracts outside this scope could introduce risks, potentially impacting overall system security.
- **Dependency on External Contracts**: The system relies on external contracts. Although integrations were reviewed, vulnerabilities or issues within these external systems could still pose risks.
- **Administrative Key Management Risks**: The system makes use of administrative keys to manage critical operations. Compromise or misuse of these keys could result in unauthorized actions and financial losses.

# Findings

## Security Review

InfiniFi Protocol



# Findings

## 3S-Protocol-M01

Unvalidated claim date parameters in **vaultRequestRedeem** leads to under-reported total assets

| | |
|---|---|
| Id | 3S-Protocol-M01 |
| Classification | Medium |
| Impact | High |
| Likelihood | Low |
| Category | Bug |
| Status | Addressed in #9afcaab. |

---

**Description**

**UpshiftFarm::vaultRequestRedeem** allows a caller with the **FARM_SWAP_CALLER** role to request a share redemption from the Upshift vault. When **lagDuration > 0**, the redemption is not instant; instead, assets become claimable at a future date identified by a **(year, month, day)** key. The function encodes this key and adds it to the **pendingClaimsKeys** set so that **UpshiftFarm::assets** can account for the pending claim when reporting total asset value.

The **_year**, **_month**, and **_day** parameters are supplied by the caller rather than derived from the vault itself. The vault's **requestRedeem** returns **(uint256 assets, uint256 claimableEpoch)**, but **vaultRequestRedeem** discards **claimableEpoch**. The vault also exposes **getWithdrawalEpoch()** which returns the correct **(year, month, day)** for the current redemption window, but this is not used either.

The integrity check **claimableAssets >= assetsOut** queries the vault's claimable amount for the caller-supplied date. If the caller passes a date corresponding to a previously existing claim that already holds sufficient **claimableAssets**, the check passes, the existing key is not duplicated in the **EnumerableSet** (since **add** is a no-op for existing keys), and the actual claim key for the new redemption is never recorded in **pendingClaimsKeys**.

As a result, **UpshiftFarm::assets** does not iterate over the new claim key and omits the newly pending assets from its total, under-reporting the farm's actual value.

#### Steps to Reproduce

1. The **FARM_SWAP_CALLER** calls **UpshiftFarm::vaultRequestRedeem** with valid **_shares** and the correct **_year**, **_month**, **_day** for epoch E1. The claim key for E1 is added to **pendingClaimsKeys**.

2. The same caller calls **UpshiftFarm::vaultRequestRedeem** again with new **_shares**, but passes the same **_year**, **_month**, **_day** (E1) instead of the actual claimable epoch E2 returned by the vault.

3. Because E1 already has **claimableAssets >= assetsOut** (from the first redemption plus the new one accumulating under E1 in the vault, or simply because E1's balance is large enough), the **AssetsIntegrityBroken** check passes.

4. **pendingClaimsKeys.add** is a no-op since the E1 key already exists. The actual E2 key is never added.

5. **UpshiftFarm::assets** iterates over **pendingClaimsKeys**, queries claimable amounts for E1 only, and does not account for the assets pending under E2. The reported total assets are lower than the actual value held by the farm.

---

### Recommendation

Remove the **_year**, **_month**, and **_day** parameters from **vaultRequestRedeem** and derive the claim date directly from the vault. Two approaches are viable:

**Option A**: Call **vault.getWithdrawalEpoch()** to obtain the correct **(year, month, day)** before requesting the redemption:

```
- function vaultRequestRedeem(uint256 _shares, uint256 _year, uint256 _month,
uint256 _day)
+ function vaultRequestRedeem(uint256 _shares)
      external
      checkSlippage
      whenNotPaused
      onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
  {
+     (uint256 _year, uint256 _month, uint256 _day,) =
IUpshiftVault(vault).getWithdrawalEpoch();
      (uint256 assetsOut,) = IUpshiftVault(vault).requestRedeem(_shares, address(this),
address(this));
      if (IUpshiftVault(vault).lagDuration() > 0) {
          uint256 claimableAssets =
              IUpshiftVault(vault).getClaimableAmountByReceiver(_year, _month, _day,
address(this));
          pendingClaimsKeys.add(encodeISODate(_year, _month, _day));
          require(claimableAssets >= assetsOut, AssetsIntegrityBroken(assetsOut,
```

```
claimableAssets));
    }
    emit VaultRequestRedeem(block.timestamp, _shares, assetsOut, _year, _month,
_day);
 }
```

**Option B**: Use the **claimableEpoch** returned by **requestRedeem** and convert it with **DateUtils.timestampToDate**:

```
- function vaultRequestRedeem(uint256 _shares, uint256 _year, uint256 _month,
uint256 _day)
+ function vaultRequestRedeem(uint256 _shares)
    external
    checkSlippage
    whenNotPaused
    onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
 {
-    (uint256 assetsOut,) = IUpshiftVault(vault).requestRedeem(_shares, address(this),
address(this));
+    (uint256 assetsOut, uint256 claimableEpoch) =
IUpshiftVault(vault).requestRedeem(_shares, address(this), address(this));
    if (IUpshiftVault(vault).lagDuration() > 0) {
+       (uint256 _year, uint256 _month, uint256 _day) =
DateUtils.timestampToDate(claimableEpoch);
        uint256 claimableAssets =
           IUpshiftVault(vault).getClaimableAmountByReceiver(_year, _month, _day,
address(this));
        pendingClaimsKeys.add(encodeISODate(_year, _month, _day));
        require(claimableAssets >= assetsOut, AssetsIntegrityBroken(assetsOut,
claimableAssets));
    }
    emit VaultRequestRedeem(block.timestamp, _shares, assetsOut, _year, _month,
_day);
 }
```

Both options ensure the claim key always corresponds to the actual claimable date determined by the vault, eliminating the possibility of referencing stale or incorrect dates. The **IUpshiftVault** interface should be updated accordingly to expose **getWithdrawalEpoch()** if Option A is chosen.

# 3S-Protocol-L01

## Missing **checkSlippage** modifier on **UpshiftFarm::vaultDeposit** allows loss of farm value through share price manipulation

| | |
|---|---|
| Id | 3S-Protocol-L01 |
| Classification | Low |
| Impact | Medium |
| Likelihood | Low |
| Category | Bug |
| Status | Addressed in #0a63276. |

### Description

**UpshiftFarm::vaultDeposit** deposits the farm's idle **assetToken** balance into the Upshift ERC-4626 vault in exchange for vault shares. Unlike **vaultRequestRedeem**, **vaultInstantRedeem**, and **vaultClaim**, the **vaultDeposit** function does not apply the **checkSlippage** modifier.

The Upshift vault exposes an **updateTotalAssets** function that allows the vault operator to adjust the reported external assets. This adjustment is bounded by **maxChangePercent**, which is currently set to **20**, representing a 20% maximum change per day. The vault's share price is derived from these reported total assets, so an **updateTotalAssets** call directly affects how many shares a depositor receives for a given amount of underlying tokens.

Because **vaultDeposit** lacks slippage protection, the vault operator (or a colluding party) can sandwich the farm's deposit by calling **updateTotalAssets** to inflate the vault's total assets immediately before the deposit, which inflates the share price and causes the farm to receive fewer shares than expected. After the deposit, a subsequent **updateTotalAssets** call can deflate total assets back, leaving the farm holding shares worth less than what was deposited. With a 20% per-day tolerance on asset changes, the potential value extraction per deposit is significant.

### Recommendation

Add the **checkSlippage** modifier to **UpshiftFarm::vaultDeposit**, consistent with the other vault interaction functions in the contract.

- **function vaultDeposit(uint256 _assets) external whenNotPaused**

```
onlyCoreRole(CoreRoles.FARM_SWAP_CALLER) {
+    function vaultDeposit(uint256 _assets) external checkSlippage whenNotPaused
onlyCoreRole(CoreRoles.FARM_SWAP_CALLER) {
        IERC20(assetToken).forceApprove(address(vault), _assets);
        uint256 shares = ERC4626(vault).deposit(_assets, address(this));
        emit VaultDeposit(block.timestamp, _assets, shares);
    }
```

# 3S-Protocol-N01

## Unused contract balance in **LiquidationFarm::liquidate** leads to suboptimal capital efficiency

| | |
|---|---|
| Id | 3S-Protocol-N01 |
| Classification | None |
| Category | Suggestion |
| Status | Acknowledged |

---

### Description

**LiquidationFarm::liquidate** enables authorized callers to pull **assetToken** from other farms via **ManualRebalancer::singleMovement**, execute an arbitrary whitelisted call (e.g., a liquidation), and smooth the resulting profit through the **PLSmoother**. The function currently requires all capital used in the liquidation to be sourced exclusively from external farms.

However, the **LiquidationFarm** contract itself may hold an **assetToken** balance from previous liquidation profits or other operations. This balance is already accounted for in **assets()** (inherited from **MultiAssetFarmV2**, which returns **ERC20(assetToken).balanceOf(address(this))**), and the vested profits from the **PLSmoother** are safe to use because the profitability check (**assetsAfter > assetsBefore + totalPulled**) guarantees that every liquidation is net-positive.

By not allowing the contract's own balance to participate in liquidations, the function forces unnecessary capital movement from other farms, reducing overall capital efficiency.

#### Steps to Reproduce

1. A liquidation is executed via **LiquidationFarm::liquidate**, generating a profit that remains in the contract as **assetToken** balance.

2. A subsequent liquidation opportunity arises.

3. The caller must pull the full required amount from external farms via the **farms** array, even though the contract already holds sufficient **assetToken** to partially or fully cover the liquidation.

4. The idle **assetToken** balance in the contract is not leveraged, resulting in unnecessary fund movement and reduced capital efficiency.

---

### Recommendation

Allow using the contract's existing **assetToken** balance by treating **address(0)** in the **farms** array as a sentinel value that signals the use of local funds instead of pulling from an external farm:

```
    uint256 totalPulled;
    for (uint256 i = 0; i < farms.length; i++) {
       require(amounts[i] != 0 && amounts[i] != type(uint256).max,
InvalidAmount(amounts[i]));
+       if (farms[i] != address(0)) {
           ManualRebalancer(manualRebalancer).singleMovement(farms[i],
address(this), amounts[i]);
           totalPulled += amounts[i];
+       } else {
+           require(i == 0);
+           require(amounts[i] <= IERC20(assetToken).balanceOf(address(this)));
+           totalPulled += amounts[i];
+       }
    }
```

This is safe because the existing profitability invariant (**assetsAfter > assetsBefore + totalPulled**) ensures the contract never loses funds during a liquidation, regardless of whether the capital originated from an external farm or from the contract's own balance.

# 3S-Protocol-N02

Incomplete **harvest** precondition in **RWAEscrowRateManager** leads to incorrect yield accounting on withdrawals

| | |
|---|---|
| Id | 3S-Protocol-N02 |
| Classification | None |
| Category | Suggestion |
| Status | Addressed in #e7a0a04. |

---

## Description

**RWAEscrowRateManager::harvest** computes accrued yield or loss for an **RWAEscrow** by reading **totalAssets** and **lastUpdatedAt**, then applying the configured annual rate over the elapsed time:

The NatSpec on line 60 warns that **harvest** must be called prior to new deposits, because both **RWAEscrow::deposit** and **RWAEscrow::withdraw** reset **lastUpdatedAt** to **block.timestamp** and mutate **tosoliditytalAssets**:

**/// @dev !!! MUST call this prior to new deposits otherwise rate calculation might get messed up.**

However, the comment omits the withdrawal case. A **withdraw** call without a preceding **harvest** produces the same incorrect accounting: **lastUpdatedAt** is reset, which erases the time window over which yield should have accrued, and **totalAssets** is reduced, so any subsequent **harvest** computes yield on the wrong (lower) principal for a shorter (reset) time period.

---

## Recommendation

Update the NatSpec to warn about both deposits and withdrawals:

```
-    /// @dev !!! MUST call this prior to new deposits otherwise rate calculation might get messed up.
+    /// @dev !!! MUST call this prior to new deposits or withdrawals otherwise rate calculation might get messed up.
```

# 3S-Protocol-N03

Cumulative **claimableAssets** query in **UpshiftFarm::vaultRequestRedeem** allows the **AssetsIntegrityBroken** check to be bypassed on subsequent requests within the same claim date

| | |
|---|---|
| Id | 3S-Protocol-N03 |
| Classification | None |
| Category | Suggestion |
| Status | Addressed in #9afcaab. |

---

## Description

**UpshiftFarm::vaultRequestRedeem** redeems vault shares through the Upshift vault and, when **lagDuration** is non-zero, validates that the resulting claimable amount is at least as large as the **assetsOut** returned by **requestRedeem**. The validation queries **getClaimableAmountByReceiver** after the redeem request has already been enqueued, which returns the **cumulative** claimable amount for **address(this)** at the specified **(_year, _month, _day)** date, not the marginal amount attributable to the current request.

When multiple **requestRedeem** calls target the same claim date, the cumulative **claimableAssets** value grows with each request while **assetsOut** only reflects the latest one. This makes the **require(claimableAssets >= assetsOut)** check trivially pass for subsequent requests, even if the vault returned fewer assets than expected for a given request.

#### Steps to Reproduce

1. The **FARM_SWAP_CALLER** calls **UpshiftFarm::vaultRequestRedeem** to redeem shares worth 100 assets, targeting claim date 2026-03-01. **assetsOut** is 100, and **getClaimableAmountByReceiver** returns 100. The check **100 >= 100** passes correctly.

2. In the same block (or any subsequent call targeting the same claim date), a second call to **UpshiftFarm::vaultRequestRedeem** redeems shares that should be worth 100 assets, but the vault only credits 50 due to slippage or rounding. **assetsOut** is 50, while **getClaimableAmountByReceiver** now returns 150 (cumulative: 100 from the first request + 50 from the second). The check **150 >= 50** passes trivially, masking the discrepancy.

---

## Recommendation

Snapshot the claimable amount before the **requestRedeem** call and compare the delta against **assetsOut**:

```
    function vaultRequestRedeem(uint256 _shares, uint256 _year, uint256 _month,
uint256 _day)
        external
        checkSlippage
        whenNotPaused
        onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
    {
+       uint256 claimableBefore;
+       if (IUpshiftVault(vault).lagDuration() > 0) {
+           claimableBefore =
+               IUpshiftVault(vault).getClaimableAmountByReceiver(_year, _month, _day,
address(this));
+       }
+
        (uint256 assetsOut,) = IUpshiftVault(vault).requestRedeem(_shares,
address(this), address(this));

        if (IUpshiftVault(vault).lagDuration() > 0) {
            uint256 claimableAssets =
-               IUpshiftVault(vault).getClaimableAmountByReceiver(_year, _month, _day,
address(this));
+               IUpshiftVault(vault).getClaimableAmountByReceiver(_year, _month, _day,
address(this))
+               - claimableBefore;

            pendingClaimsKeys.add(encodeISODate(_year, _month, _day));
            require(claimableAssets >= assetsOut, AssetsIntegrityBroken(assetsOut,
claimableAssets));
        }

        emit VaultRequestRedeem(block.timestamp, _shares, assetsOut, _year, _month,
_day);
    }
```

# 3S-Protocol-N04

## Ignored return value from **singleMovement** in **LiquidationFarm::liquidate** leads to reverted liquidations when pulled amount is capped

| | |
|---|---|
| Id | 3S-Protocol-N04 |
| Classification | None |
| Category | Suggestion |
| Status | Addressed in #79cc7dc. |

### Description

**LiquidationFarm::liquidate** pulls funds from one or more source farms via **ManualRebalancer::singleMovement** and then executes a whitelisted external call to perform a liquidation. After the call, the function verifies that the farm's **assets()** increased by more than **totalPulled**, ensuring the operation was profitable.

**ManualRebalancer::_singleMovement** caps the actual transferred amount to the destination farm's **maxDeposit** and returns the effective amount moved. However, **LiquidationFarm::liquidate** discards this return value and instead accumulates the requested **amounts[i]** into **totalPulled**:

```
ManualRebalancer(manualRebalancer).singleMovement(farms[i], address(this), amounts[i]);
totalPulled += amounts[i];
```

When **maxDeposit** is lower than the requested **amounts[i]**, fewer tokens are actually transferred than recorded. This inflates **totalPulled** above the real amount received, which causes the subsequent profit check (**assetsAfter > assetsBefore + totalPulled**) to revert with **UnprofitableOperation** even when the liquidation was genuinely profitable. The result is that any liquidation where at least one source farm's transfer gets capped by **maxDeposit** becomes impossible to execute.

### Recommendation

Capture the return value of **ManualRebalancer::singleMovement** and use it instead of the requested amount when accumulating **totalPulled**.

```
- ManualRebalancer(manualRebalancer).singleMovement(farms[i], address(this),
amounts[i]);
- totalPulled += amounts[i];
+ uint256 pulled = ManualRebalancer(manualRebalancer).singleMovement(farms[i],
address(this), amounts[i]);
+ totalPulled += pulled;
```

# 3S-Protocol-N05

Deterministic hash in **PLSmoother::smoothProfit** causes revert when two same-size liquidations occur in the same block

| | |
|---|---|
| Id | 3S-Protocol-N05 |
| Classification | None |
| Category | Suggestion |
| Status | Acknowledged |

## Description

**PLSmoother::smoothProfit** computes a unique identifier for each smoothing item by hashing **(block.timestamp, msg.sender, receiptTokenProfit, duration)**. This identifier is used as a key in the **smoothingItems** set, which enforces uniqueness. When the same caller triggers two liquidations within the same block that produce identical **receiptTokenProfit** and **duration** values, the resulting hash is the same for both calls. The second call will revert because the key already exists in the set, causing a lost liquidation opportunity with no functional justification.

This is a realistic scenario in protocols where liquidation bots or keepers batch multiple liquidations in a single block for gas efficiency. If two eligible positions happen to share the same profit and duration parameters, one of them becomes unliquidatable until the next block, delaying the protocol's ability to process bad debt or realize profits.

## Recommendation

Include **smoothingItems.length()** in the hash computation to guarantee uniqueness across entries created in the same block by the same caller with identical parameters.

```
- bytes32 id = keccak256(abi.encode(block.timestamp, msg.sender,
receiptTokenProfit, duration));
+ bytes32 id = keccak256(abi.encode(block.timestamp, msg.sender,
receiptTokenProfit, duration, smoothingItems.length()));
```

# 3S-Protocol-N06

Redundant positive profit check in **LiquidationFarm::liquidate** leads to unnecessary gas overhead

| | |
|---|---|
| Id | 3S-Protocol-N06 |
| Classification | None |
| Category | Suggestion |
| Status | Addressed in #bed3cf7. |

## Description

**LiquidationFarm::liquidate** executes an atomic liquidation operation by pulling funds from other farms, calling a whitelisted external target, and verifying that the resulting asset balance increased. The profit verification section contains three sequential checks on lines 156–163:

```
require(
    assetsAfter > assetsBefore + totalPulled, UnprofitableOperation(assetsBefore,
totalPulled, assetsAfter)
);
uint256 assetTokenProfit = assetsAfter - assetsBefore - totalPulled;
require(assetTokenProfit > 0, InsufficientProfit(1, assetTokenProfit));
require(assetTokenProfit >= minProfit, InsufficientProfit(minProfit,
assetTokenProfit));
```

The first **require** already enforces that **assetsAfter > assetsBefore + totalPulled**, which mathematically guarantees that **assetTokenProfit** (computed as **assetsAfter - assetsBefore - totalPulled**) is strictly greater than zero. The second **require(assetTokenProfit > 0, ...)** is therefore redundant and can never revert independently of the first check. The third check enforces the caller-supplied **minProfit** threshold and remains necessary since **minProfit** can be zero.

## Recommendation

Remove the redundant **require(assetTokenProfit > 0, ...)** statement on line 162, as the preceding **UnprofitableOperation** check already guarantees a positive profit.

```
    uint256 assetsAfter = assets();
```

```
        require(
            assetsAfter > assetsBefore + totalPulled,
UnprofitableOperation(assetsBefore, totalPulled, assetsAfter)
        );
        uint256 assetTokenProfit = assetsAfter - assetsBefore - totalPulled;
-       // enforce > 0 profit to avoid "liquidation" calls that would perform other
-       // operations such as token approvals
-       require(assetTokenProfit > 0, InsufficientProfit(1, assetTokenProfit));
        require(assetTokenProfit >= minProfit, InsufficientProfit(minProfit,
assetTokenProfit));
```

# 3S-Protocol-N07

Missing **assetToken** and **vault** asset consistency check in **UpshiftFarm** constructor allows misconfigured deployments

| | |
|---|---|
| Id | 3S-Protocol-N07 |
| Classification | None |
| Category | Suggestion |
| Status | Addressed in #12ce142. |

## Description

**UpshiftFarm** is a yield-bearing farm that deposits its **assetToken** into an Upshift ERC-4626 vault. The constructor accepts both **_vault** and **_assetToken** as independent parameters and forwards **_assetToken** to the parent **MultiAssetFarmV2** constructor, while storing **_vault** as an immutable. However, **UpshiftFarm::constructor** never verifies that the provided **_assetToken** matches the vault's actual underlying asset returned by **ERC4626(_vault).asset()**.

If the farm is deployed with an **_assetToken** that differs from the vault's underlying asset, **UpshiftFarm::vaultDeposit** will approve and attempt to deposit the wrong token into the vault. Depending on the vault implementation, this would either revert on every deposit or, worse, silently succeed with unexpected behavior. The **UpshiftFarm::assets** function would also return incorrect totals, as it converts vault shares to assets using a vault whose underlying token does not match the farm's accounting denomination. Since **vault** and **assetToken** are both immutable, the only remediation after a misconfigured deployment would be redeploying the contract.

## Recommendation

Add a check in the constructor to ensure that **_assetToken** matches the vault's underlying asset.

```
 constructor(address _core, address _vault, address _assetToken, address
_accounting)
     MultiAssetFarmV2(_core, _assetToken, _accounting)
 {
+     require(_assetToken == ERC4626(_vault).asset(), "asset token mismatch");
     // set default slippage tolerance to 99.5%
     maxSlippage = 0.995e18;
     vault = _vault;
```

```
}
```